Yad2 Real Estate Market Tracker — Ra'anana 4-Room Apartments

A fully automated system that scrapes, tracks, and visualises the Ra'anana 4-room apartment market on Yad2. It runs twice daily, detects every new listing, price change, and removal, and presents everything in a live interactive dashboard.

How It Works

```
 Yad2 Website
      │
      ▼
yad2_scraper.py  ──────────────────────────────────────────────────────
│  1. Opens Yad2 feed page (18+ pages) via Playwright browser          │
│  2. Extracts all listing cards (price, street, rooms, floor, sqm)    │
│  3. Fetches real posting date for each listing via in-browser fetch  │
│  4. Detects new listings, removed listings, price changes            │
│  5. Computes market stats (avg price, absorption, DOM, survival...)  │
│  6. Saves everything to data files                                   │
└──────────────────────────────────────────────────────────────────────┘
      │
      ▼  writes to ──►
┌──────────────────────────────────────────────────────────────────────┐
│  DATA FILES                                                          │
│  yad2_raanana_4rooms.json / .csv   — current listings snapshot       │
│  yad2_prev_run.json                — previous run snapshot           │
│  yad2_first_seen.json              — URL → first-seen date           │
│  yad2_date_posted.json             — URL → real Yad2 posting date    │
│  yad2_history.csv                  — market stats time-series        │
│  yad2_neighborhood_history.csv     — neighbourhood price history     │
│  yad2_dom_log.csv                  — removed listings + DOM          │
│  yad2_run_log.csv                  — log of every scraper run        │
└──────────────────────────────────────────────────────────────────────┘
      │
      ▼  reads from ──►
yad2_dashboard.py  (Streamlit)
      │
      ▼
Browser dashboard at http://localhost:8501
```

Running the Scraper

```
python yad2_scraper.py
```

- Takes ~5–10 minutes (18 pages + date fetching for all listings)
- Scheduled to run automatically at **08:00** and **20:00** daily
- Output is printed to console: pages scraped, dates fetched, changes detected

---

Opening the Dashboard

Option A — double-click:
`2יד דשבורד פתח.bat`

Option B — terminal:

```
streamlit run yad2_dashboard.py
```

Opens at `http://localhost:8501`

File Reference

File	Updated by	Purpose
`yad2_scraper.py`	(you run it)	Main scraper script
`yad2_dashboard.py`	(you edit it)	Streamlit dashboard
`2יד דשבורד פתח.bat`	—	Shortcut to launch dashboard
`yad2_raanana_4rooms.json`	scraper	Full current listings with all fields
`yad2_raanana_4rooms.csv`	scraper	Same data as CSV (for Excel etc.)
`yad2_prev_run.json`	scraper	Snapshot of the previous run — diff against this to detect changes
`yad2_first_seen.json`	scraper	Dictionary: URL → date we first saw this listing
`yad2_date_posted.json`	scraper	Cache: URL → real Yad2 posting date (so pages aren't re-fetched)
`yad2_history.csv`	scraper	One row per run: market-wide stats over time
`yad2_neighborhood_history.csv`	scraper	One row per neighbourhood per run: price trends
`yad2_dom_log.csv`	scraper	Every removed listing with its days-on-market
`yad2_run_log.csv`	scraper	Every run: status, counts, errors

Listing Data Fields

Each record in `yad2_raanana_4rooms.json` (and `.csv`) has:

Field	Example	Description
`price`	`₪ 2,900,000`	Asking price as shown on Yad2
`street`	`הגלבוע`	Street name
`info`	`דירה, לב הפארק, רעננה`	Apt type, neighbourhood, city
`rooms`	`4`	Number of rooms
`floor`	`3`	Floor number
`size_sqm`	`100`	Size in square metres
`url`	`https://www.yad2.co.il/...`	Direct link to listing
`date_posted`	`2025-07-10`	Real date the seller posted on Yad2

The dashboard also computes two derived columns at display time:

Column	How computed
`תאריך הופעה` (appearance date)	`date_posted` if available, else `first_seen` date
`ימים בשוק` (days on market)	`today − date_posted`

History & Statistics Files

`yad2_history.csv`

One row per scraper run with market-wide stats:

Column	Description
`timestamp`	Date and time of run
`count`	Total active listings
`avg_price`	Average asking price
`median_price`	Median asking price
`min_price`	Cheapest listing
`max_price`	Most expensive listing
`avg_price_per_sqm`	Average price per m^2

Column	Description
`price_cuts`	Number of listings that lowered price
`price_rises`	Number of listings that raised price
`avg_cut_pct`	Average % of price cuts
`absorption_rate`	% of listings removed this run (sold/withdrawn)
`survival_30d`	% of listings that were still active after 30 days
`avg_dom`	Average days-on-market for listings removed this run

`yad2_neighborhood_history.csv`

One row per neighbourhood per run:

Column	Description
`timestamp`	Date and time of run
`neighborhood`	Neighbourhood name
`avg_price`	Average price in that neighbourhood
`avg_price_per_sqm`	Average price per m^2
`count`	Number of active listings

`yad2_dom_log.csv`

One row per listing that was removed (sold or taken down):

Column	Description
`removed_date`	Date we noticed it was gone
`url`	Listing URL
`first_seen`	Date we first saw this listing
`dom_days`	Days on market (removed_date − first_seen)
`price`	Last known price
`neighborhood`	Neighbourhood
`size_sqm`	Size
`apt_type`	Apartment type

`yad2_run_log.csv`

One row per scraper run:

Column	Description
`timestamp`	When the run started
`status`	`success` or `error`
`pages_scraped`	Number of feed pages scraped
`total_listings`	Total listings found
`new_listings`	New listings since last run
`removed_listings`	Listings gone since last run
`price_cuts` / `price_rises`	Price movements
`avg_cut_pct`	Average price cut %
`avg_price` / `median_price` / etc.	Market stats for that run
`absorption_rate`	% removed this run
`survival_30d`	30-day survival rate
`avg_dom`	Average DOM of removed listings
`error`	Error message if status = error

Dashboard Sections

Section	What it shows
Sidebar	Filters: price range, size, apt type, rooms, floor, neighbourhood. Live pill showing new/removed/price-changes since last run.
KPI strip	Active listings · Avg price · Median price · Min price · Price per m^2
Market trends	Price per m^2 over time · Listing count over time · Price range (min/avg/median/max) · Price cuts vs rises per run
Time & absorption	Absorption rate · 30-day survival · Avg DOM (removed listings) · Avg price cut depth · DOM distribution histogram · DOM by neighbourhood
Price analysis	Price bands (<2M / 2–3M / 3–4M / 4–5M / 5M+)
Changes since last run	Tabs: New listings · Removed listings · Price changes (with before/after)
Current snapshot	Price distribution histogram · Apt type pie chart · Supply by neighbourhood · Size vs price scatter · Avg price by neighbourhood

Section	What it shows
Full table	All active listings with search, max-days filter, max-price filter. Columns: price, street, neighbourhood, type, rooms, floor, size, **date posted**, **days on market**, link.
Run log	History of every scraper run with all stats

Date Extraction — How It Works

Getting the real posting date from Yad2 requires three attempts in order:

1. **React fiber state** — reads dates directly from React component memory on the feed page (zero extra requests). Works when Yad2 exposes the date in component props.
2. **Network response interception** — captures Yad2's own API JSON responses during page load and extracts dates from them automatically.
3. **In-browser fetch fallback** — for any listing still missing a date, the scraper uses the already-loaded browser (which has valid session cookies) to fetch each individual listing page via JavaScript `fetch()`. This bypasses Yad2's bot-protection (ShieldSquare). Parses `פורסם ב DD/MM/YY` from the HTML. Runs in batches of 20 in parallel.
4. **First-seen safety net** — if all else fails, uses the date we first noticed the listing in our tracker.

Fetched dates are permanently cached in `yad2_date_posted.json` so they are never re-fetched on subsequent runs.

Technical Requirements

```
 Python 3.10+
playwright (pip install playwright && playwright install chromium)
streamlit
pandas
plotly
```

Automation Schedule

The scraper is configured to run automatically at **08:00** and **20:00** every day via Windows Task Scheduler. Each run takes ~5–10 minutes. The dashboard footer shows the last update time.